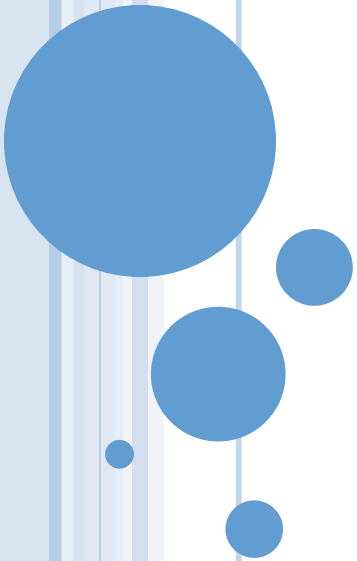


The Technical Debt Metaphor

Davide Falessi, Ph.D.

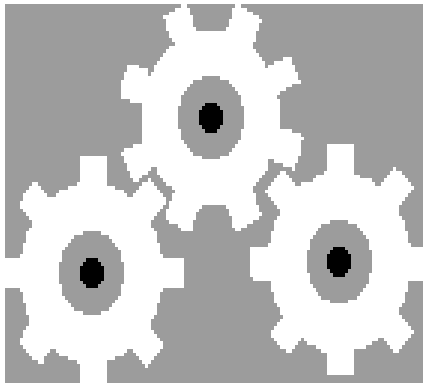
Agenda

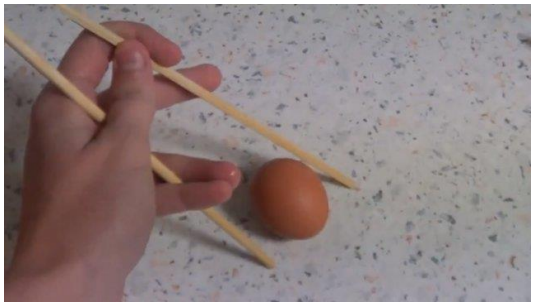
- Aim
- Definitions
- Practical Considerations
- Tool Support
  - Strengths
  - Limits
- Industrial Case Study
- Research Directions
- References

Aim

- Provide an overview about Technical Debt.
- Why?

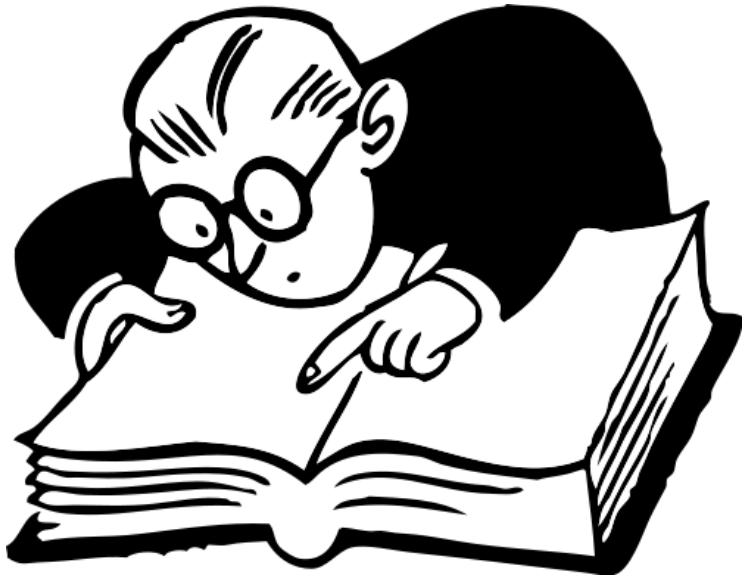








DEFINITIONS



Introduction

- TD is the application of **financial** concepts to **software** engineering domain.

- TD is a metaphor:
  - Pros: **widely** applicable.
  - Cons: highly prone to **misuses**.

Terminology

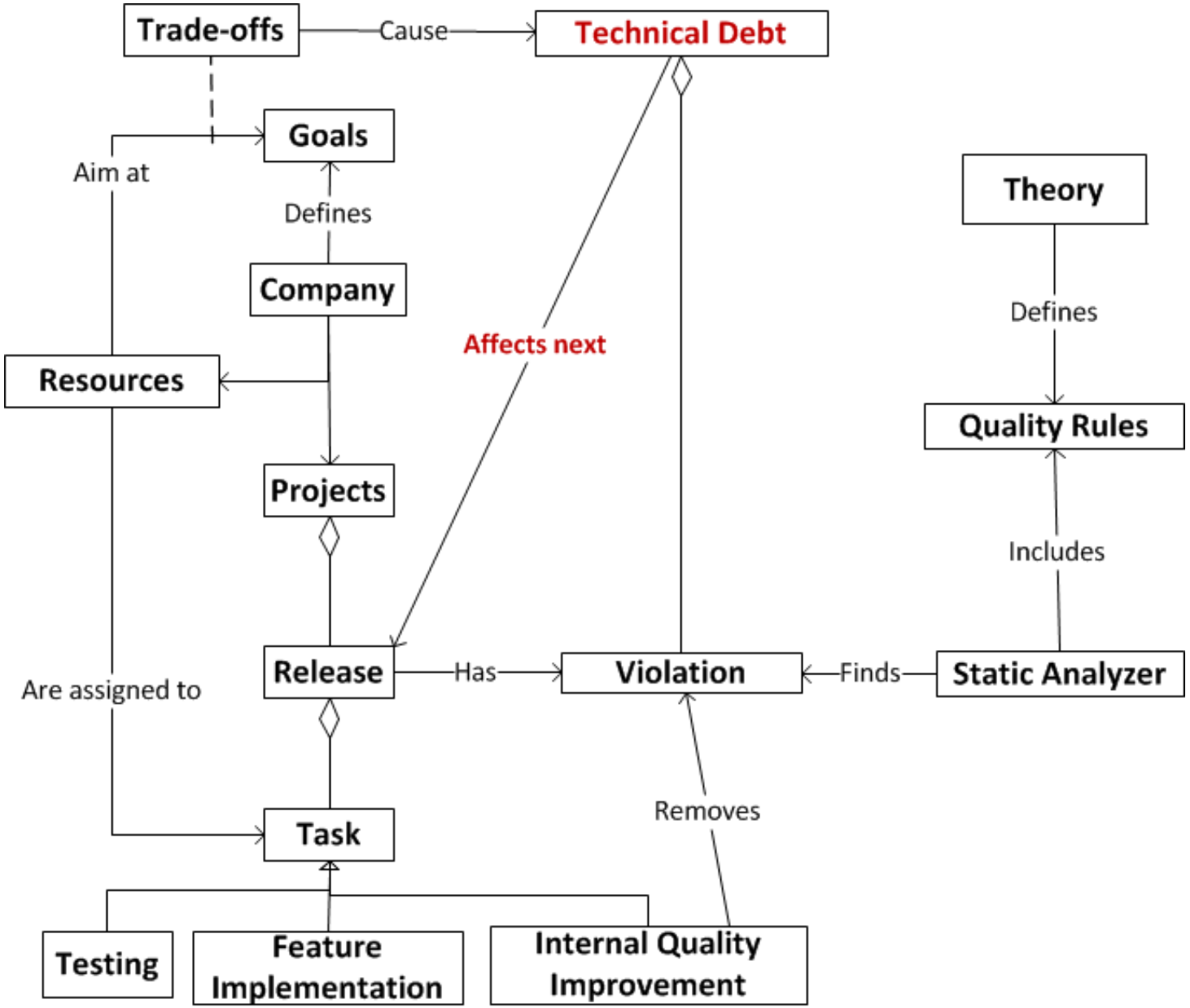
- **Quality Rule:** An empirically validated software engineering principle dictating how the code should be. Examples of quality rules are: high comments density, low code complexity, short conditional statements, etc.
- **Violation:** A portion of a code that is not compliant with a specific rule.
- **Defect:** A problem in the code that requires some effort to be fixed. In this context, a quality rule is *not* a defect. Quality rules relate to the *internal quality* of the software product and are *invisible* to the user. Vice versa, defects relate to the external quality of the code.

Possible sources

- Technical Debt (TD) can be seen as the result of an optimization for the **short-term** which leads to **long-term** handicaps.
  - Examples: low comments density, code smell, code clone, etc.

- TD can:
  - Emerge **organically** as every system while evolving, increases complexity.
  - Be **opportunistically** chosen; e.g., “let’s release now, we’ll deal with it later on”.

My view


Trade-offs
Cause
Technical Debt
Goals
Aim at
Defines
Company
Affects next
Theory
Defines
Resources
Quality Rules
Projects
Includes
Are assigned to
Release
Has
Violation
Finds
Static Analyzer
Removes
Task
Testing
Feature Implementation
Internal Quality Improvement

Main concepts

- TD consists of two important concepts:
  - **Principal**: the cost of eliminating the debt.
  - **Interest**: the penalty to be paid in the future if the debt is not eliminated.

    - Example: a high complex module could require significant effort to be refactored (**principal**), however, if not refactored, it could slow down development speed (**interest**).

TD vs. Maintainability

- TD is very close to **Maintainability**. TD adds **time and economic criteria** in decision-making.

- The concept of TD **proved** to be useful:
  - Large organizations (e.g., Siemens, Lockheed Martin, etc.) have explicitly introduced TD, in some form or another, in their software development process, as something to **identify, value, and take into consideration** while planning iterations and releases.

PRACTICAL CONSIDERATIONS



Debt can be a good thing

- TD can organically **expire**.
  - All projects have a lifetime. Financial debt?
- Contracts may not include maintainability.



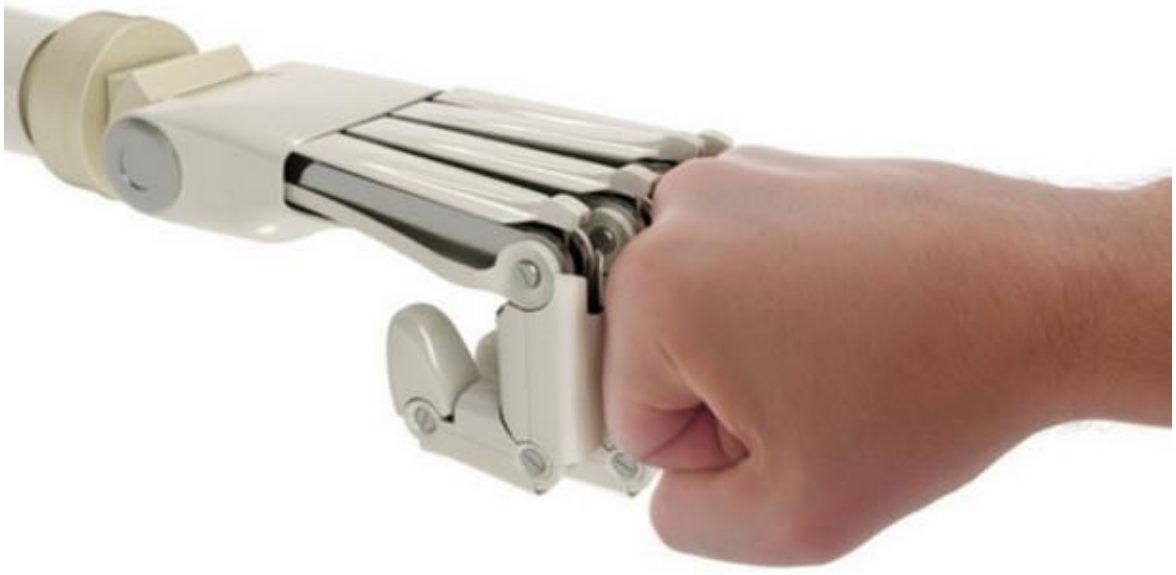
Imperfection is the norm

- **All projects** have some TD.
  - In a world of finite resources, **imperfection** is the norm.
- Shall I eliminate TD?

Yes… but not always!

- The success of a project depends also (especially) on business goals: cost, time-to-market and customers' satisfaction.
- TD must be managed, monitored, and analyzed. **Always**!

TOOL SUPPORT



Tool – State of the art: SonarQube

- **Open Source.**
- **Maintained** by a big community.
- **Largely used** in industry.
- More than **20 programming languages.**
- More than **50 plugins.**
- More than **1200 quality rules** like **code smells** (e.g., god classes) and **coding style** (e.g., comment density).
- **Fast and easy to use.**
- Several **visualizations techniques.**
- **Demo:** https://sonarcloud.io/explore/projects

SonarQube limits

- Principal **quantification**:
  - Unclear nature.
  - Hard to customize.
  - Good metric.
- Interest **quantification**:
  - Unclear nature.
  - Hard to customize.
  - Bad metric.

Case Study



Context and motivations

- Industrial problems:
  - Too **many** violations to manage.
  - Rules are neither **validated** nor **customized** to the application context.

- Aim:
  - Understanding **if**, **which**, and **how much**, quality rules are really important to prevent defects during software maintenance.

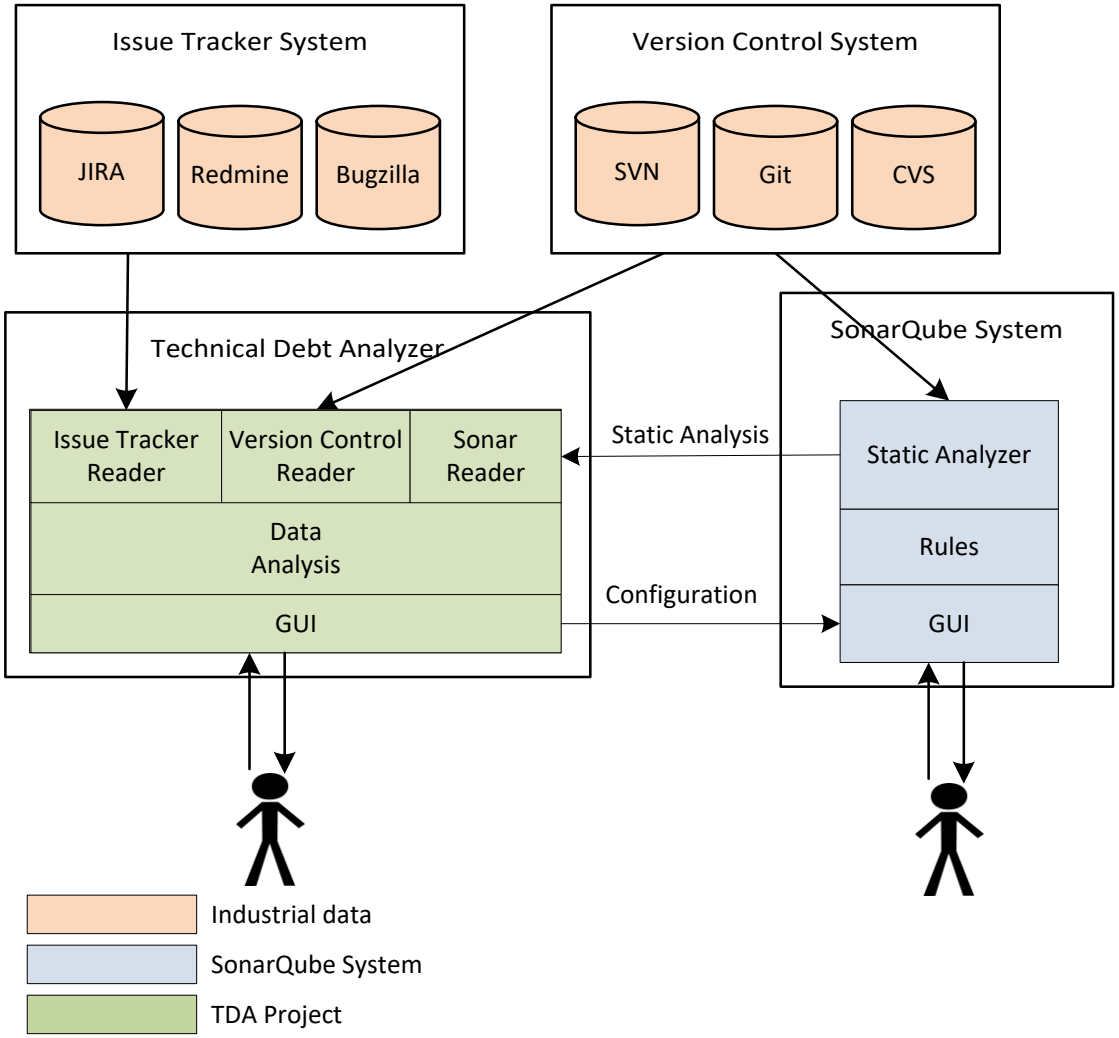
Research questions

1. Are quality rules important to prevent defects?
2. Does the importance vary among quality rules?
3. Do rule information support testing activity? (i.e., Defect prediction?)

Research methodology

- Analysis of CMMI level 5 company **dataset**.
- A **survey** with 6 experts developers.
- Adoption of 5 **Machine Learning** techniques (WEKA) to predict defects.

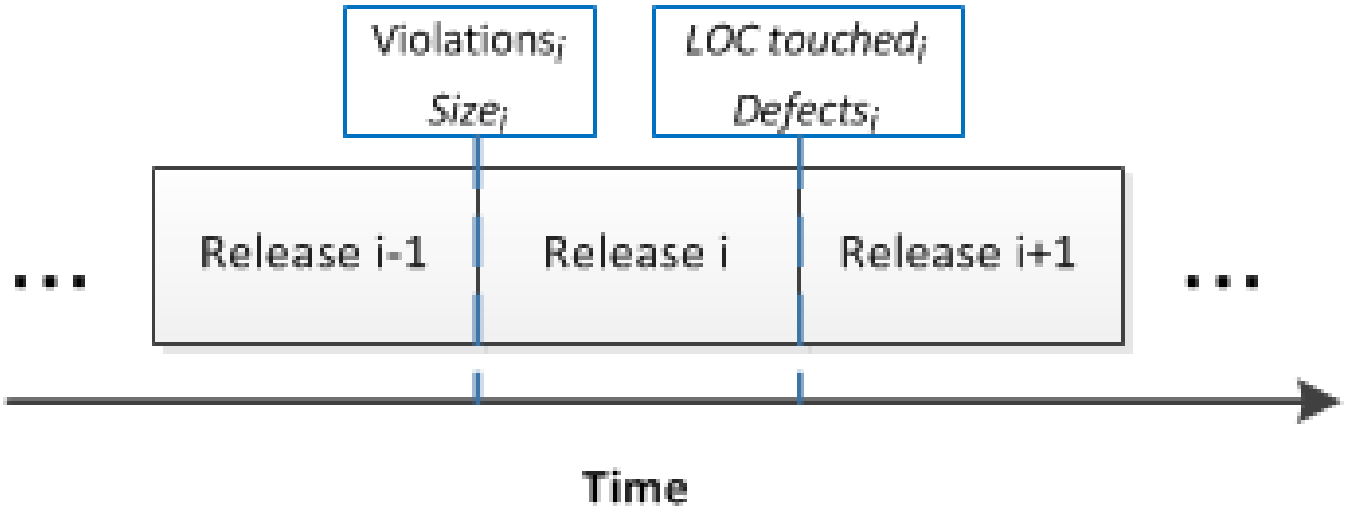
Research Methodology: Tools



RQ1: Are quality rules important to prevent defects?

- Null Hypothesis:
  - H_{01} = When evolving classes with different levels of violation density, there is no difference in the number of defects introduced per LOC touched.

- Measurement Approach:

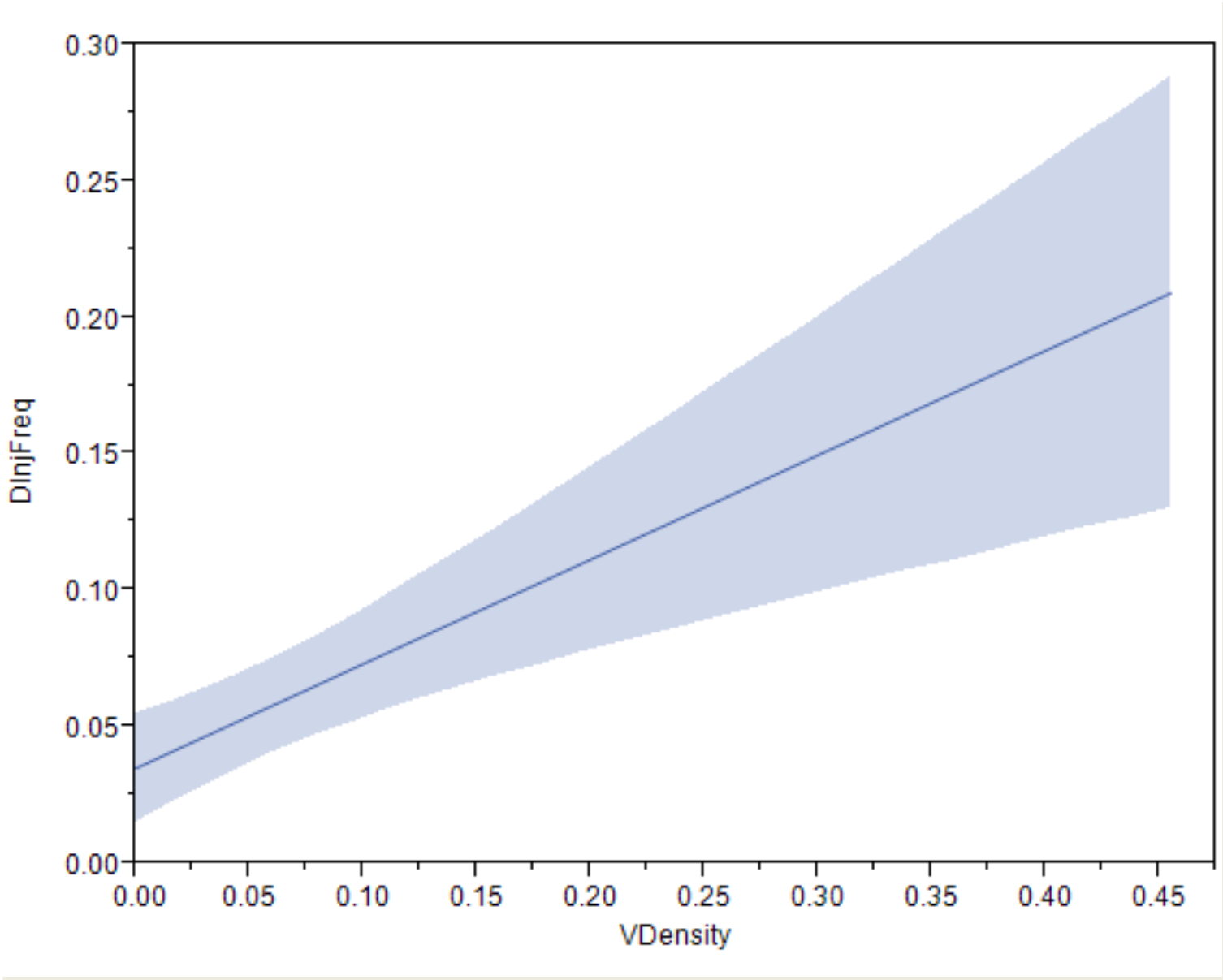


RQ1: Are quality rules important to prevent defects?

Percentile	Size	VC	LOC touched	Defects
100%	11709	213	2360	57
90%	519	43.6	120.3	7
75%	277	12	46	3
50% (Median)	141	0	11	1
25%	78.75	0	3	0
10%	40.1	0	1	0
0%	14	0	1	0
Mean	283.03	12.21	62.65	2.6

Characteristics of the **classes** of the analyzed projects

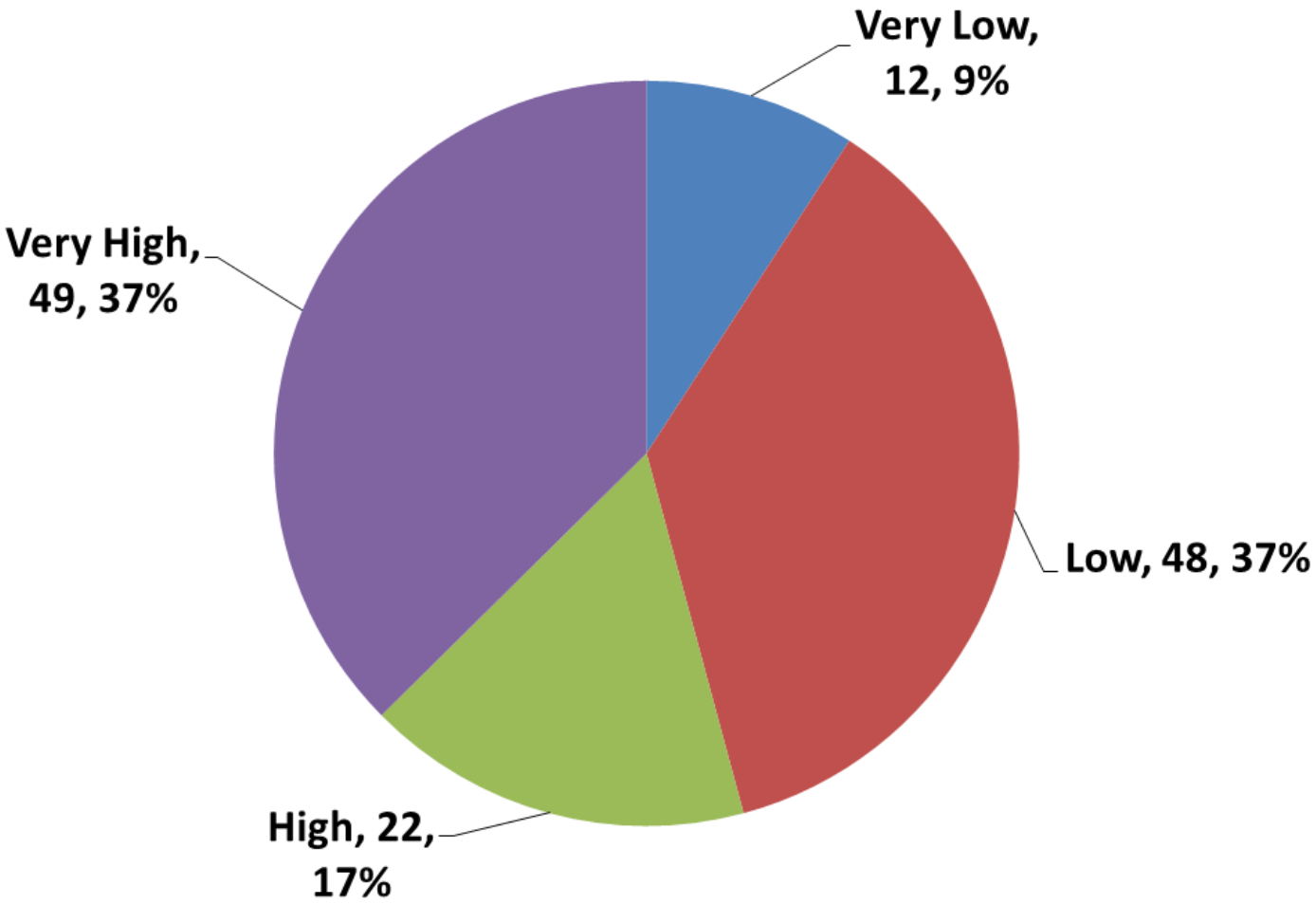
RQ1: Are quality rules important to prevent defects?



RQ2: Does the importance vary across quality rules?

- Null Hypothesis:
  - H_{02} = All rules have the same importance.
- Measurement approach:
  - Survey of 6 developers at Keymind
    - *How much does violating this rule negatively influence the maintainability?*
    - "*Very High*", "*High*", "*Low*", "*Very Low*", "*I didn't understand the meaning of the rule*", and "*I'm unsure about the importance of the rule*"

RQ2: Does the importance vary across quality rules?



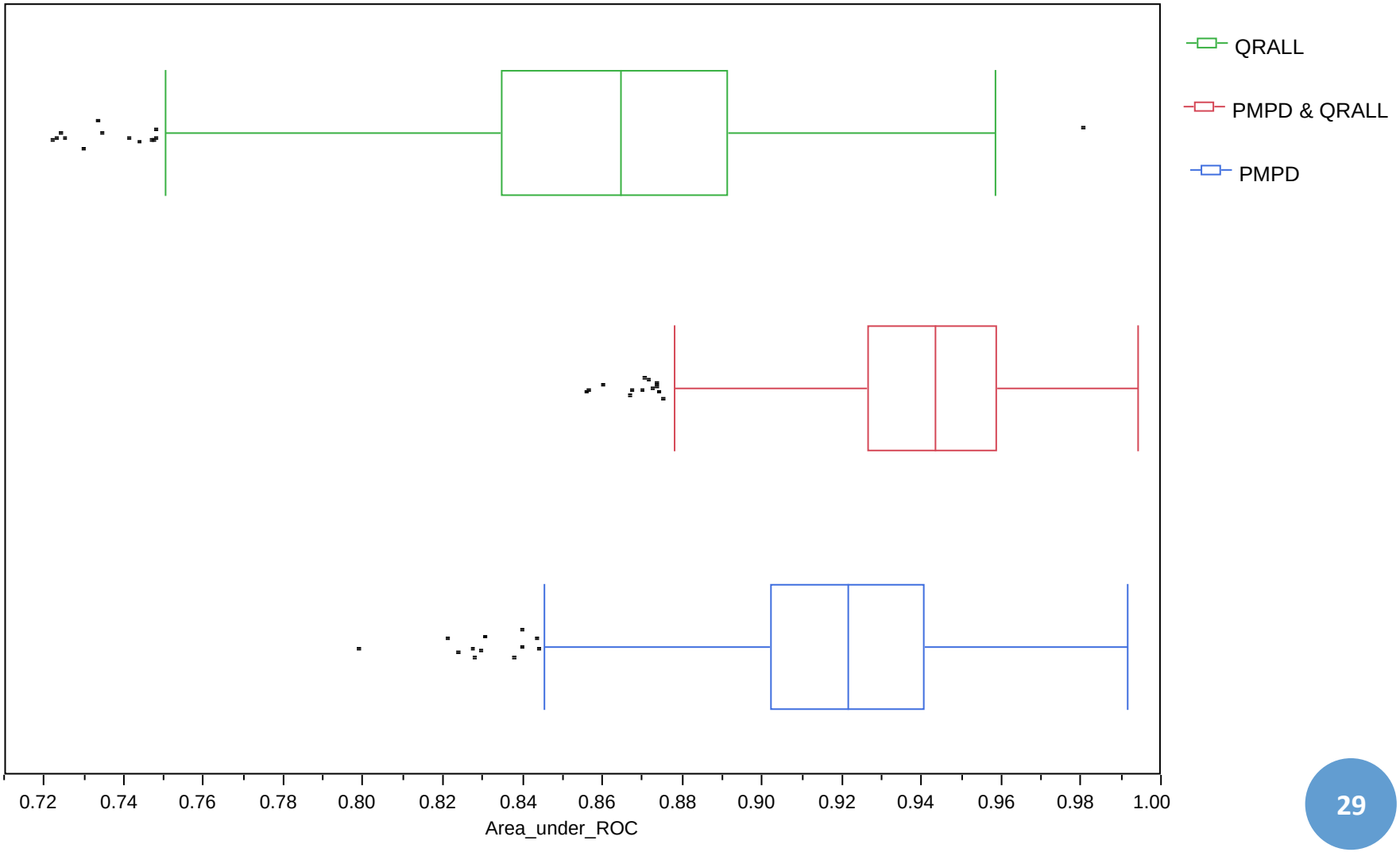
RQ3: Do rule information support testing activity? (i.e., Defect prediction?)

- Methodology – predicting the classes with defects and the number of defects.
  - **Accuracy**:
    - Quality rules impact number of defects **if** they improve the defect prediction accuracy when compared to standard metrics.
  - **Selection**:
    - Quality rules impact number of defects **if** they are selected as (useful) input for prediction (regardless of the resulting prediction accuracy).

RQ3: Do rule information support testing activity? (i.e., Defect prediction?)

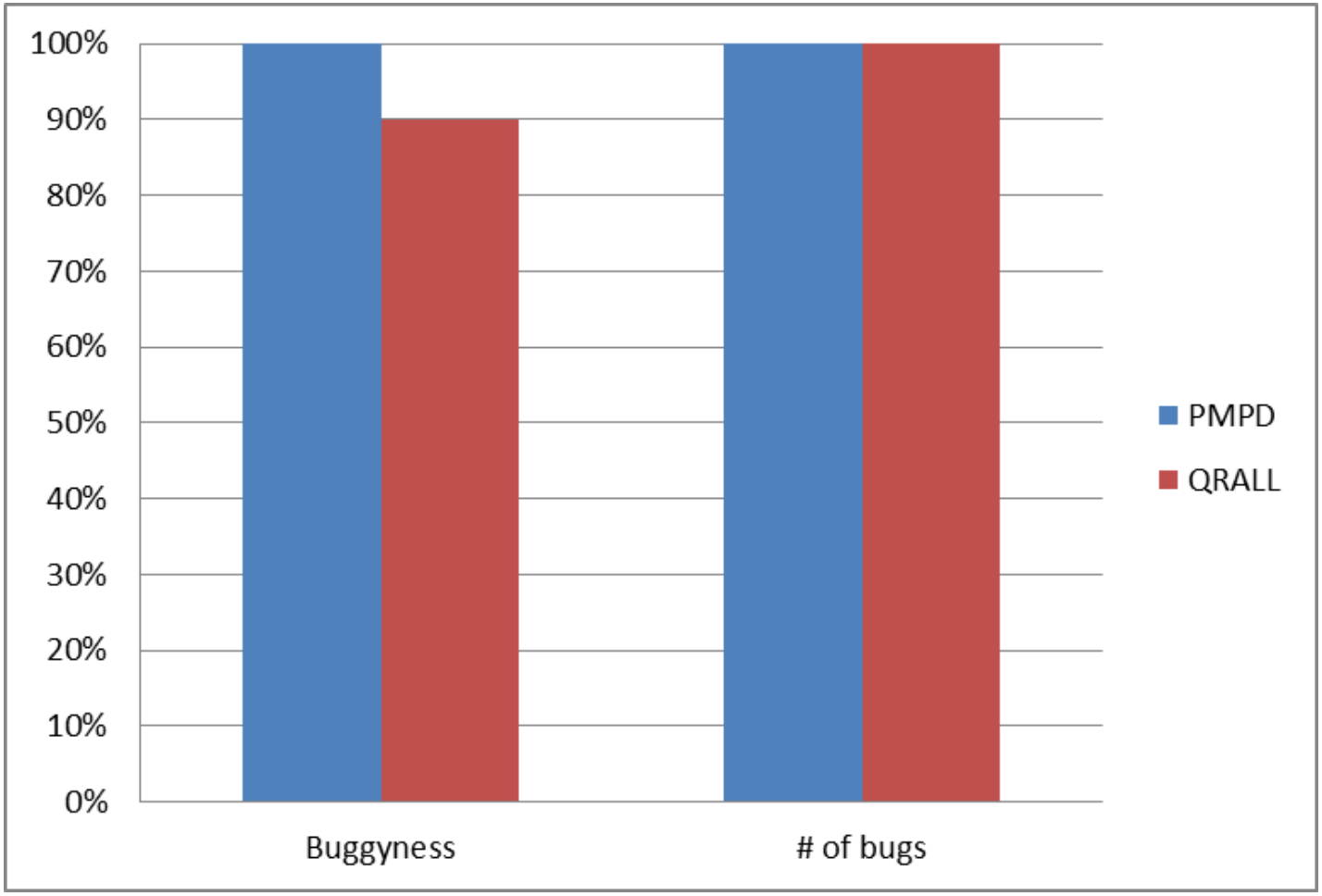
- **Measurement** approach:
  - 5 machine learning techniques:
    - reporting only "Bagging"
  - 100 times 10-fold cross-validation:
    - standard way to validate machine learning techniques in SE domain.

RQ3: Do rule information support testing activity? (i.e., Defect prediction?)



*Distribution of **accuracy** in predicting **buggy classes** when using different sources of information*

RQ3: Do rule information support testing activity? (i.e., Defect prediction?)



*Frequency of **attribute selection** among different source of information when predicting buggy classes and number of bugs*

Research Directions



Research Directions: 1 Robustness in data analysis

- Industrial repositories of historical data are **never** 100% accurate.
- *Data analysis techniques* are currently evaluated, compared, and used **assuming** the dataset is 100% accurate.
- The **robustness** of data analysis techniques, to the inaccuracy of the underlying data, should be:
  - Evaluated.
  - Improved.
  - Reported.

Research Directions: 2 Merging heterogeneous information

- Experts opinions is always **valuable** but never completely **accurate**.
- Historical analysis is always **valuable** but never completely **accurate**.

- **Combining** these two sources of information:
  - How?
  - When?
  - Improved accuracy?

Research Directions: 3 Developing more knowledge

- Exploring the **disadvantages** of fixing quality rules.
- Building a body of knowledge of **causal** relations rather than of **correlations**.

Research Directions: 4 NLP for change proneness prediction

- Predicting the **change proneness** of a class by analyzing the requirements via natural language processing techniques.
  - Do you recall the Ferrari example?

Research Directions: 5 Leveraging economic models

- Real-option, portfolio management, …

Acknowledgements

- Alex Voegele, Graduate student at Fraunhofer CESE

- Technical Debt gang:
  - Philippe Kruchten, UBC
  - Ipek Ozkaya, SEI
  - Robert Nord, SEI

References on Technical Debt

- *D. Falessi,* P. Kruchten, R. L. Nord, I. Ozkaya, "Technical debt at the crossroads of research and practice: report on the 5th international workshop on managing technical debt." ACM SIGSOFT Software Engineering Notes 39(2): 31-33, 2014
- P. Kruchten, R. L. Nord, I. Ozkaya, and *D. Falessi*, "Technical debt: towards a crisper definition report on the 4th international workshop on managing technical debt." ACM SIGSOFT Software Engineering Notes, Vol. 38, Is. 5 Pages: 51-54. 2014
- *D. Falessi*, M. Shaw, F. Shull, K. Mullen, M. Stein, "Practical Considerations, Challenges, and Requirements of Tool-Support for Managing Technical Debt", 4th International Workshop on Managing Technical Debt (MTD 2013), co-located with ICSE, San Francisco, USA, 2013.
- F. Shull, *D. Falessi*, C. Seaman, M. Diep, L. Layman, "Technical Debt: Showing the Way for Better Transfer of Empirical Results", Perspectives on the Future of Software Engineering, pp 179-190, 2013.